

Day 13: Introduction to Heaps (*Complete Binary Trees)

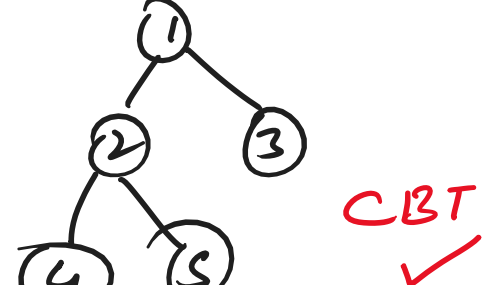
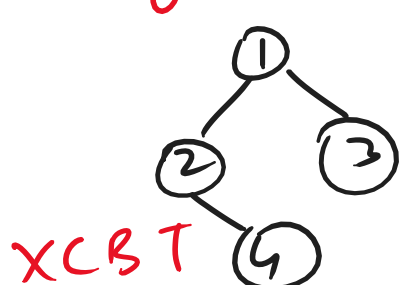
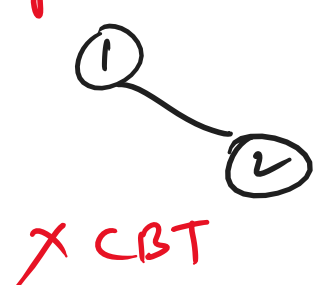
- * Greedy Algos
- * Graphs
- * Sliding Window Problems

Question (kth)

* max * min
{ range of values }

CBT: ① Insertions will always happen left to right in each level & there can be leaf nodes available

② We cannot insert a node to the right leaving an empty left node.



* Why or when Heaps come into existence?

Sort $\rightarrow n \log n$

* Give me the largest value

* Give me the smallest value

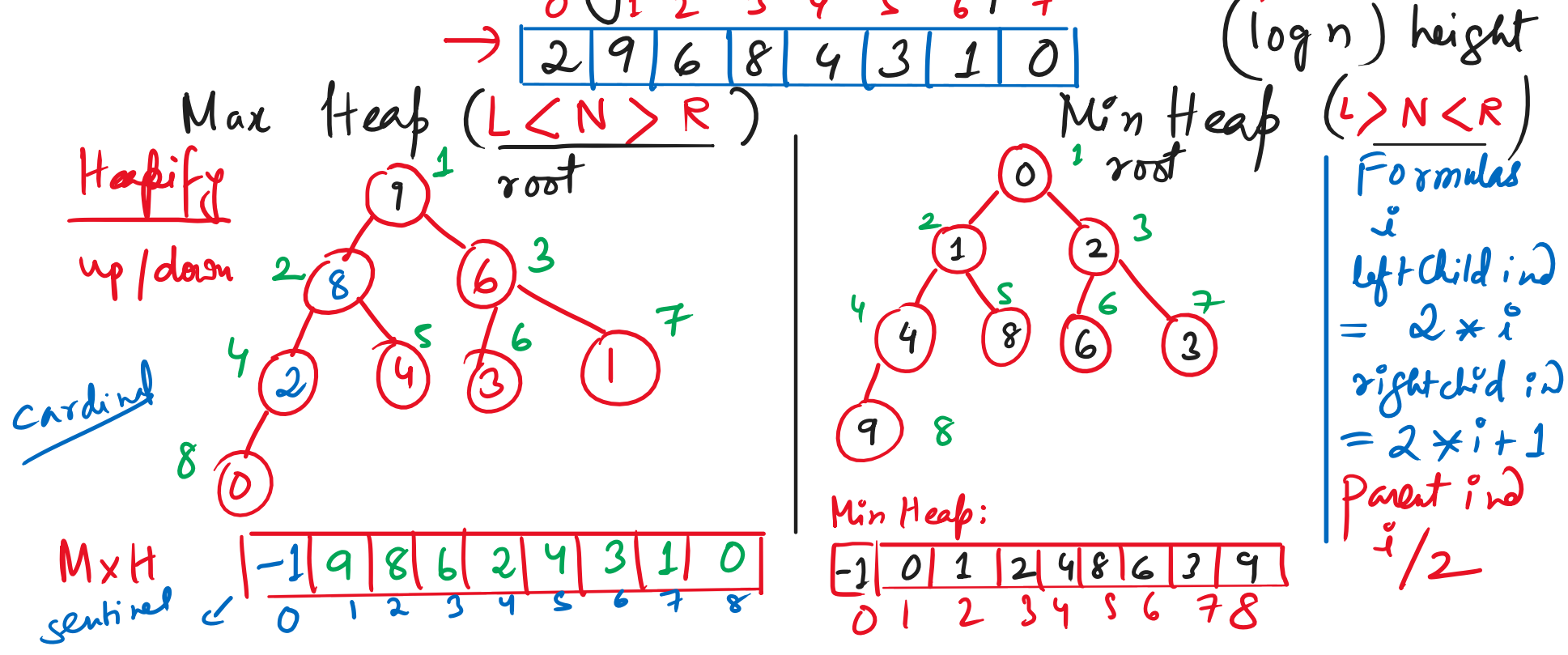
very slow $\{ n \log n + O[1] \}$

$O(1)$ Time $\Rightarrow$ Heaps

Max Heap

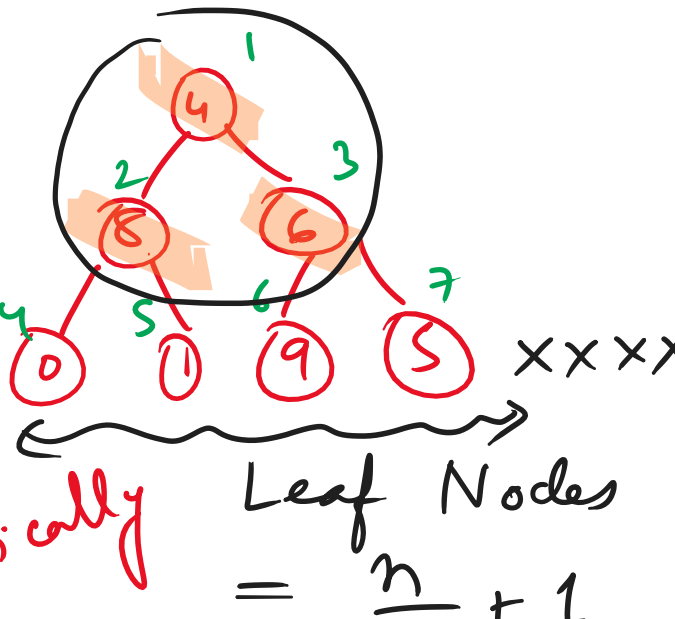
Min Heap

* Convert an array into MxHeap / MnHeap (CBT)



array = 4, 8, 6, 0, 1, 9, 5 $\rightarrow$ normal array

1, 2, 3 non-leaf nodes are converted to heap, & the rest gets converted automatically

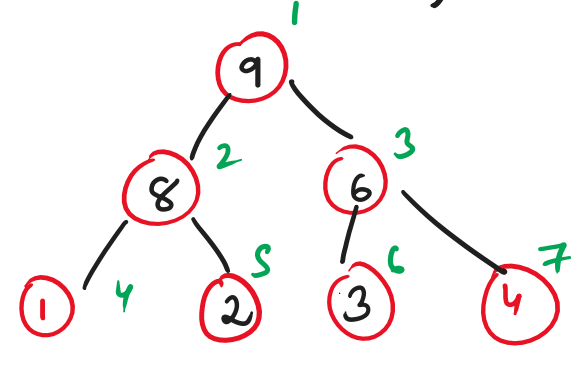


Convert the given array into a heap:

* Only the non-leaf nodes are considered. $n-4 = 3$

array = [2, 8, 3, 1, 9, 4, 6] size = 7

MaxHeap: $\rightarrow (L < N > R)$ (CBT)

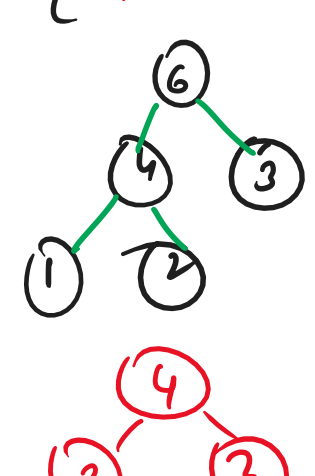
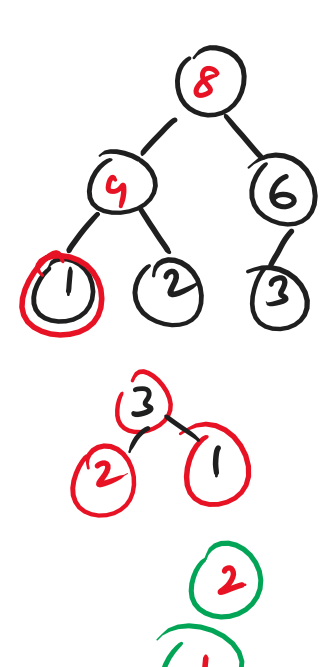


Pseudo Code: $\rightarrow$

- * swap (arr[i], arr[size])
- * size --
- * heapify index 1

Max Heap: $\rightarrow$ Repeat till finished { Recursion }

1	2	3	4	5	6	7
9	8	6	1	2	3	4
4	8	6	1	2	3	9
8	4	6	1	2	3	9
3	4	6	1	2	8	9
6	4	3	1	2	8	9
2	4	3	1	6	8	9
4	2	3	1	6	8	9
1	2	3	4	6	8	9
3	2	1	4	6	8	9
1	2	3	4	6	8	9
2	1	3	4	6	8	9
1	2	3	4	6	8	9



While (size > 1)
n nodes for log n

kth largest element from a given array: $\rightarrow$

③ { 19, 21, 4, 9, 3, 7 } k = 3
reverse (min heap) efficiency $n \log n + O[n-3]$

① Insert elements into heap
② Pop elements if size > k
③ The top element will be the kth largest

array $\Rightarrow$ 4, 9, 2, 1, 3, 0

kth Smallest $\rightarrow k=3$

Max Heap: $\rightarrow$ (TLE) **

- ① Insert all elements into the Heap
- ② Remove max elements > k size
- ③ The top element will be kth smallest

In this example 2 is the 3rd Smallest